

Ever started a project with Claude Code and wondered...

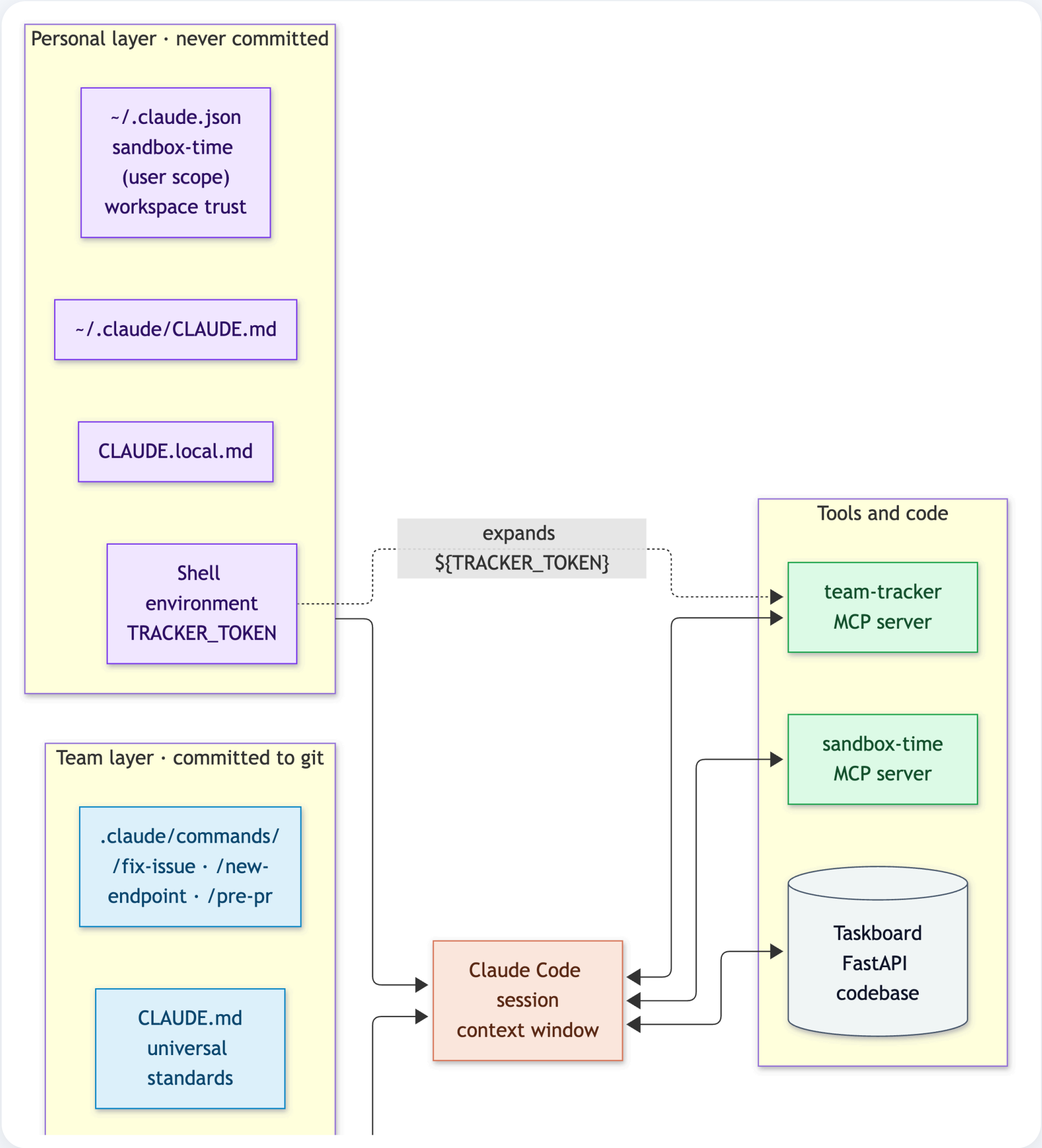
| Where do the team's instructions go?

| How do I customize things for myself without breaking everyone else's standards?

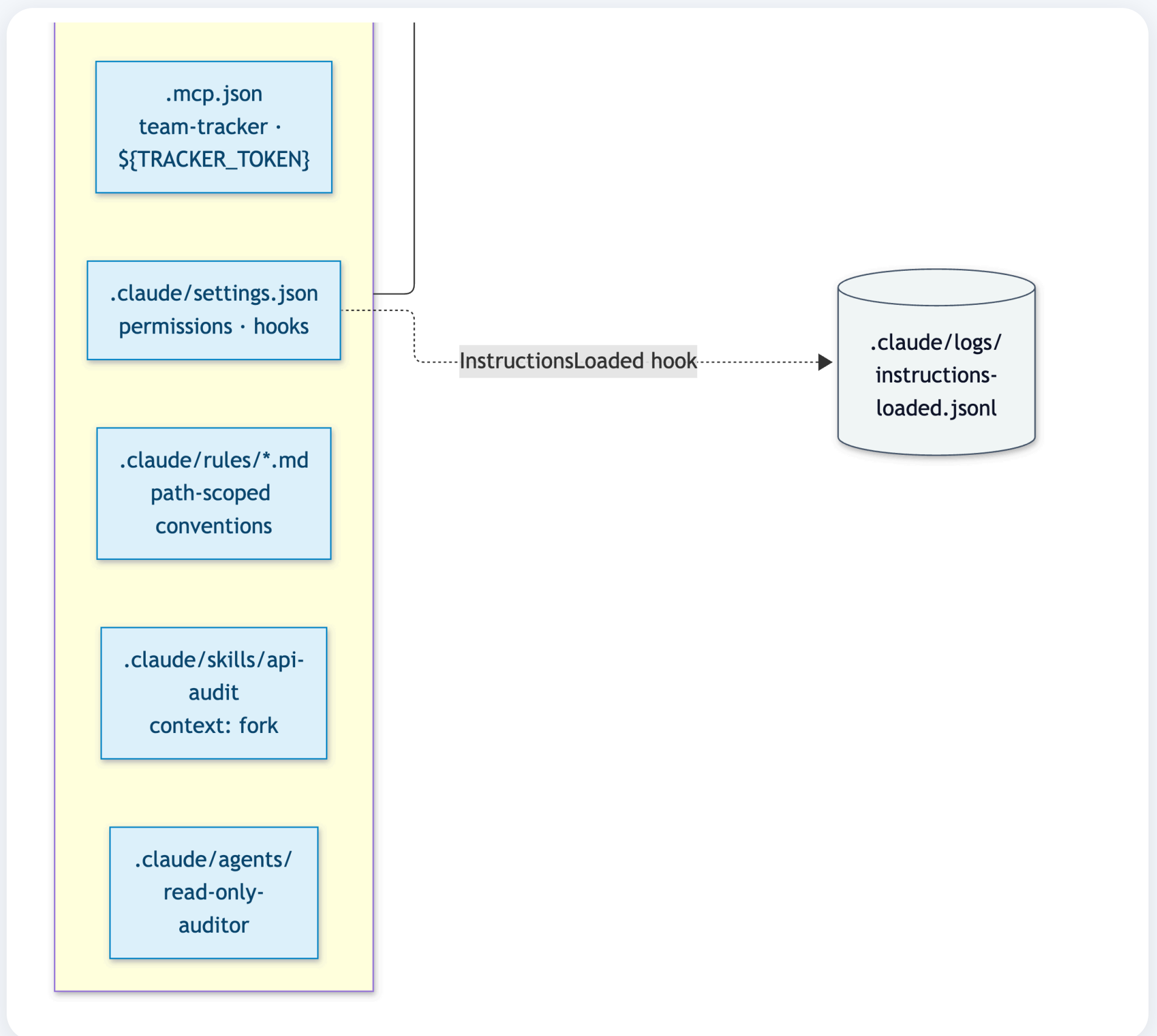
| When should Claude plan first instead of just writing code?

I built a project to answer exactly that. Swipe →

Two layers, one session



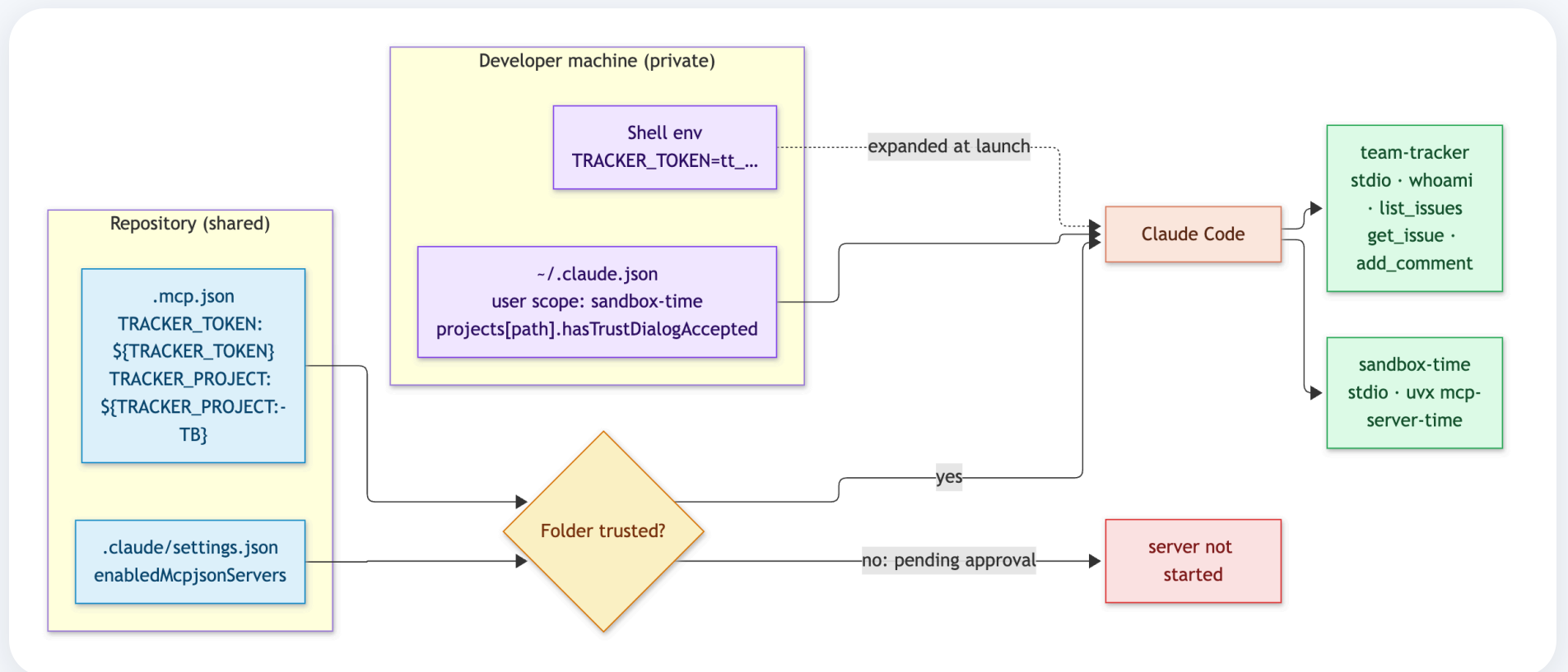
Team layer: committed to git



Shared standards, path-scoped rules, slash commands, a read-only audit skill and the team's MCP server. **Everyone gets it on clone.**

SURPRISE 1

A freshly cloned repo can't grant itself anything



Until you trust a folder, Claude Code ignores that repo's permissions, hooks and MCP approvals.

The team file holds `${TRACKER_TOKEN}`. Your real token stays in your shell environment and is expanded at launch.

Skills: what really restricts a tool

2 allowed-tools doesn't lock anything out

It only pre-approves tools. The real restriction is the subagent's own tool list.

3 The forked audit did the reading

It read the entire API layer, and the main conversation made **zero tool calls**.

Result: a read-only audit skill that runs in its own separate context and keeps your main session clean.

Measured over 8 runs

One-line bug — just do it

Same fix, twice the cost with planning.

Library migration — optional

Same quality either way. Planning only adds a review checkpoint.

New feature with open questions — plan first

The plan put every design decision in front of me before any code existed. The direct run made the same calls quietly and added an abstraction nobody asked for.

MY TAKEAWAY

Plan mode doesn't write better code.

It moves decisions to where they're still cheap to change.

Code, diagrams, tests and a step-by-step playbook:

github.com/Jagoul/claude-for-software-development

Found this useful? Repost so your team sees it, and tell me how you structure Claude Code in yours.